Software Development Project — G&G City · Lab Backlog Sprint 1

LAB S2: Transitions + The Screen — Aws Al-Ghamdi

Topic: A Horizontal Slider with CSS Transforms + Promises

Learning outcomes

By the end of this lab, students will be able to:

- Build a 16:9 responsive screen with `aspect-ratio`
- Slide between pages with `transform: translateX()` and a CSS transition
- Return a Promise that resolves when the transition ends

🎯 Problem Scenario

Each station from `lessons.json` becomes one page of a long horizontal strip. The screen shows one page at a time and slides to the next one while the cat walks.

- **S1 gives you the backgrounds, P1 gives you `stations` (with `title` and `background`)**
- **S3 will write: `cat.walk(); await scenes.goTo(1); cat.idle();` so `goTo` must wait for the slide**

Your files: `scenes/scenes.js, scenes/test.html`

Required Outputs

Build:

1. **The screen: 16:9, rounded corners, hides everything outside it**
2. **`createScenes(container, stations)` builds one station per item, each with a title and an empty `.gg-stage`**
3. **`goTo(i)` slides and resolves when finished; `next()`, `prev()`, `current()`, `stage(i)`**
4. **`test.html` with Previous / Next buttons**

Code is given only for the hard parts. Parts marked 📝 **Note** are easy: follow the note and write them yourself. Test with **Live Server** in VS Code (`fetch` does not work when you double-click a file).

Model Answer:

Task 1: The screen + track (hard part)

The screen is a window; the track is a long strip behind it. `direction: ltr` matters because our page is Arabic (rtl), which would reverse the strip.

```
.gg-screen  { position: relative; width: 100%; aspect-ratio: 16 / 9; overflow: hidden;
border-radius: 14px; background: #1e1d1a; direction: ltr; }   /* ltr: page may be rtl */
.gg-track   { display: flex; height: 100%; transition: transform .8s cubic-bezier(.65, 0,
.35, 1); }
.gg-station { flex: 0 0 100%; position: relative; background: center / cover no-repeat; }
```

Task 2: Building the stations

> 📝 **Note (no code needed):** Inject the CSS once. Create `div.gg-screen` > `div.gg-track`. For each station create a `section.gg-station` with `backgroundImage` set from `station.background`, a `.gg-title` (use `textContent`, not innerHTML) and an empty `.gg-stage` at the bottom (absolute, flex, items at the bottom). Keep the stage elements in an array for `stage(i)`.

Task 3: goTo that waits (hard part)

The fallback timer protects you when `transitionend` never fires (e.g. the tab is in the background).

```
// Hard part: slide, and resolve only when the slide has finished
function goTo(i) {
  const target = Math.max(0, Math.min(stations.length - 1, i));   // never past the ends
  if (target === index) return Promise.resolve(index);
  index = target;
  return new Promise(resolve => {
    const done = () => { clearTimeout(fallback); track.removeEventListener('transitionend',
done); resolve(index); };
    const fallback = setTimeout(done, 1000);          // in case transitionend never fires
    track.addEventListener('transitionend', done);
    track.style.transform = `translateX(${-100 * index}%)`;
  });
}
```

> 📝 **Note (no code needed):** `next()` is `goTo(index + 1)`, `prev()` is `goTo(index - 1)`, `current()` returns `index`.

Task 4: test.html

> 📝 **Note (no code needed):** Fetch `../data/lessons.json`. Until there are more stations, push two copies of the first station with different titles to see the slide. Background paths in JSON are relative to

> `index.html`, so in the test page prefix them with `../`.

Expected Output



Sliding to the next station…



…arrived, cat and box moved

Submission

```
git pull
git add scenes/
git commit -m "إضافة الانتقال الأفقي بين المحطات"
git push
```